

LM Gateway — One API, Every Provider, No Downtime Lost

A self-hosted, message-queue-based gateway for resilient,
provider-agnostic LLM infrastructure.

The integration tax compounds

[SYS.GW.02]

FRAGMENTED PROTOCOLS

Every provider speaks a different language. Gemini, OpenRouter, Ollama — each requires its own request format, auth scheme, and error handling.

RATE LIMIT FAILURES

Free-tier and production rate limits cause failed requests mid-workflow. There's no built-in strategy to route around them or retry gracefully.

DOWNTIME = LOST WORK

When a provider goes down, requests vanish. There's no persistence layer, no retry queue — just silent failure and lost progress.

REPEATED INTEGRATION EFFORT

The same adapter logic, retry code, and error handling gets rebuilt in every project. Teams never stop paying this integration tax.

One gateway. Every provider. Zero rebuilds.

[SYS.GW.03]

One Canonical API

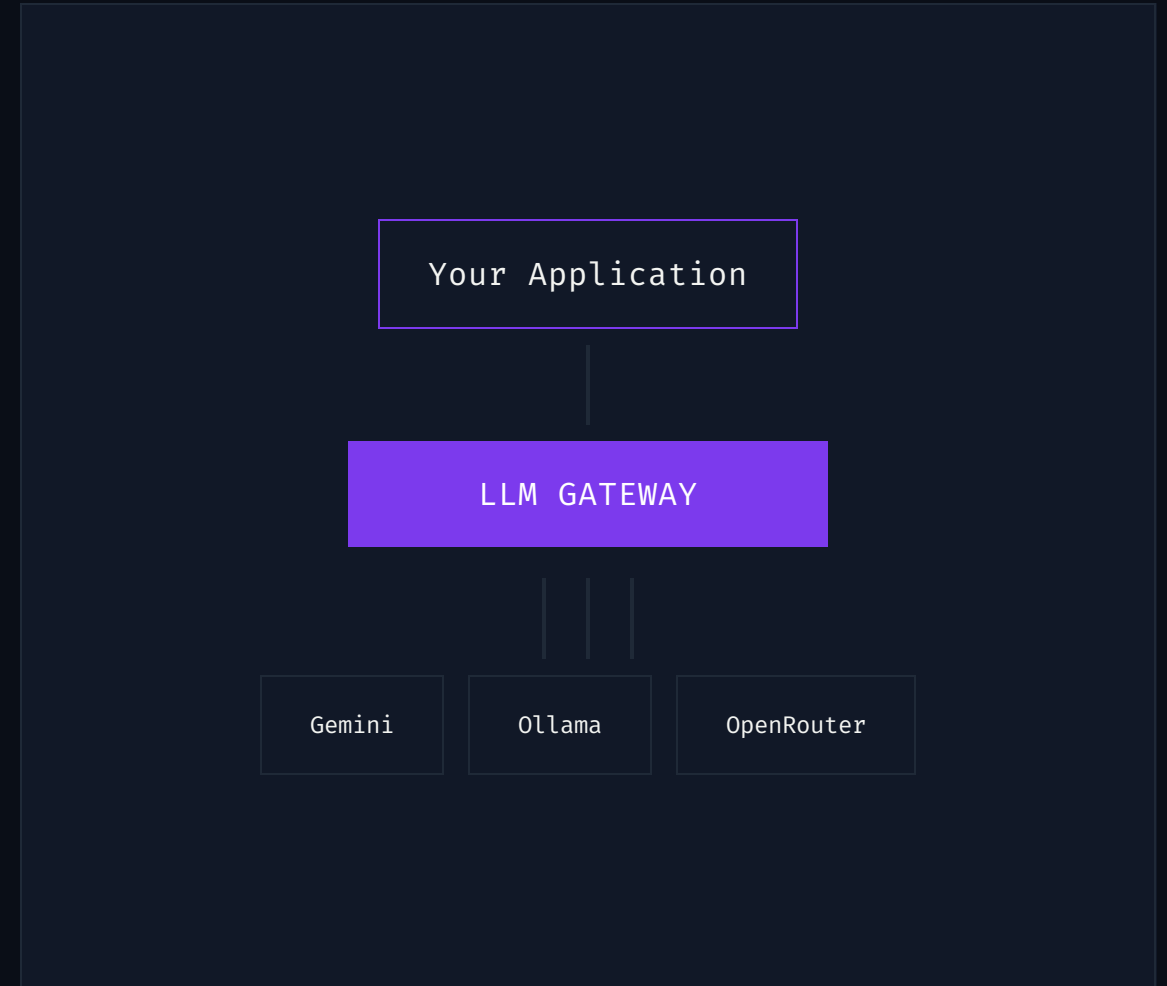
Clients call a single, stable endpoint regardless of which provider handles the request. No provider-specific shapes leak into your codebase.

Persistent Async Queue

Requests survive provider downtime. RabbitMQ persists jobs until they complete, so work is delayed, not destroyed.

Self-Hosted Microservice

Deploy it once, reuse it everywhere. LLM access becomes an internal infrastructure concern — not a per-project integration task.



Who this is built for

[SYS.GW.04]

THE IDEAL USER

Backend developers and engineering teams who want a self-hosted, provider-agnostic LLM microservice — without owning retries, rate limits, and provider-specific integration code themselves.

If your team is building more than one LLM-powered feature, or plans to, this is your infrastructure layer.

YOU'LL RECOGNIZE YOURSELF IF...

- You've rewritten LLM integration code across multiple projects.
- Provider downtime has silently broken your application.
- You're managing multiple API keys and rate-limit strategies by hand.
- You want to switch models or providers without touching application code.
- You need a single observability point for all LLM traffic.

Two execution modes, one routing core

[SYS.GW.05]

SYNC: Low-latency, interactive

- 1 POST /api/sync
Blocking request with messages & params
- 2 Pydantic Validation
Schema enforcement & type coercion
- 3 Router & Adapter
Translate params to provider-specific shape
- 4 Provider SDK Call
Direct call to Gemini/Ollama/OpenRouter
- 5 Immediate Response
Returns 200 OK with generated content

ASYNCR: Resilient, durable

- 1 POST /api/async
Non-blocking request, returns Job ID
- 2 RabbitMQ Producer
Job persisted in 'tasks' durable queue
- 3 Worker Consumer
Independent process picks up job
- 4 Result Storage
Outcome saved to SQLite result store
- 5 GET /{job_id}
Client polls for completed result

The stack is intentionally boring

[SYS.GW.06]

Layer	Technology	Role
API Framework	FastAPI	HTTP handling & OpenAPI docs
Validation	Pydantic v2	Strict schema enforcement
Message Broker	RabbitMQ	Durable async job queue
Result Store	SQLite + SQLAlchemy	Persistent job tracking
Package Manager	uv	Fast, reproducible environments

A canonical contract keeps clients stable

[SYS.GW.07]

200 OK

Request fulfilled. Check 'warning' flag for dropped params.

422 Unprocessable

Pydantic validation error — malformed request body.

429 Rate Limit

Upstream provider limit hit. Gateway can route around this.

502 Provider Failure

Upstream provider is down. Async jobs will persist.

CANONICAL REQUEST

```
{
  "provider": "gemini",
  "model": "gemini-2.5-flash",
  "messages": [
    { "role": "user", "content": "..." }
  ],
  "parameters": {
    "temperature": 0.7,
    "max_tokens": 500
  }
}
```

SYNC RESPONSE

```
{
  "code": 200,
  "warning": false,
  "message": {
    "role": "assistant",
    "content": "..."
  }
}
```

Four commitments shape every decision

[SYS.GW.08]

Provider-Agnostic

Clients interact with one stable contract — never a specific provider's API shape. Switching providers is a configuration change, not a code change.

Best-Effort Execution

The gateway favors generating a response over failing outright. Non-essential constraints are handled transparently — the caller always knows what happened.

Stable External Contract

Adding a new provider never requires clients to update their integration. The gateway's API is the only contract your team maintains.

Status Separation

HTTP status codes reflect gateway outcomes. A separate application-level result field describes what actually happened with the LLM request.

What teams actually gain

[SYS.GW.09]

1

API Contract

One canonical interface for all providers, maintained once.

3+

Providers

Gemini, OpenRouter, Ollama — and growing.

0

Lost Requests

Async queue persists through provider downtime.

Next: make resilience measurable

[SYS.GW.10]



Job Tracking

Structured, queryable job records with full status and error history.



Failure Handling

Distinct handling for timeouts, rate limits, and provider outages.



Horizontal Scale

API and worker layers scale independently with per-provider queuing.



Throughput Pooling

Route across a pool of keys to maximize combined rate-limit headroom.